Application Layer Specification

Command Definitions, Authorization, and Business Logic
for the NFC Smart Lock Protocol

Version 0.1 – Draft

July 24, 2026

Abstract

This document specifies the application layer for the NFC smart lock system. It defines the plaintext command and response formats, business logic (lock actuation, authorization policy), and the abstract interface that the session layer calls into. The application layer has no knowledge of cryptography, key material, session state, NFC framing, or transport mechanics – it receives decrypted plaintext from the session layer, acts on typed commands, and returns plaintext responses. This document was rewritten from the original combined specification after the cryptographic session content was extracted into a standalone *Session Layer Specification* (`smartlock_session_layer.pdf`).

Contents

1 Scope and Design Assumptions

This specification governs what happens *inside* an established secure session: command parsing, authorization decisions, lock actuation, and response generation. The session layer establishes the secure channel; the application layer uses it.

The following scope decisions are in effect for this revision:

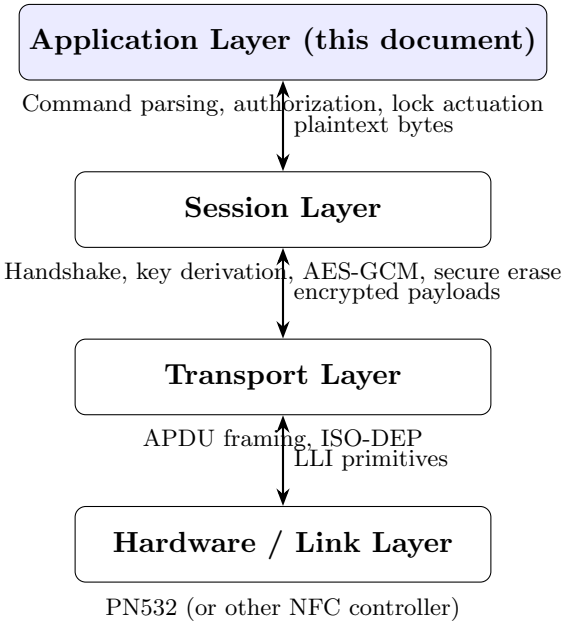
- **Authorization model:** Authentication is currently treated as equivalent to authorization – any device that can prove possession of a provisioned Ed25519 identity key is granted access. No access control list, role, or revocation mechanism is defined in this revision. See Section 6.
- **Command set:** A single command (`CMD_UNLOCK`) is defined. Future commands (OTA trigger, BLE command passthrough, ACL management) are scoped in Section 6.
- **Lock actuation:** The command to actuate the physical lock mechanism is treated as a platform-specific call; this document defines only the logical decision to actuate, not the hardware interface.

1.1 Non-Goals

This specification does not cover: cryptographic session establishment (handled by the session layer), transport framing (handled by the transport and hardware/link layer documents), provisioning workflows, firmware update security, physical tamper response, or the physical lock mechanism interface.

2 Layer Model

The application layer sits atop the session layer. It is called by the session layer with decrypted plaintext and returns plaintext responses. It never sees encrypted data, never holds key material, and has no awareness of which transport carried the session.



2.1 Abstract Interface

The application layer exposes a single entry point invoked by the session layer when a decrypted payload arrives:

Function	Called By	Behavior
`on_command`(plaintext, session_context) → response_plaintext	Session Layer	Parses the plaintext byte payload into a typed command. Dispatches to the appropriate handler. Returns plaintext response bytes, or an empty response if no reply is needed. The `session_context` supplies the authenticated peer identity (Ed25519 public key) so authorization decisions can be made.

The `session_context` carries only what the application layer needs to authorize commands – the authenticated peer identity and any session-level metadata. It does not expose keys, nonces, or cryptographic state.

```
struct session_context {
    peer_identity_pk: [u8; 32],   // peer's Ed25519 public key
    // reserved for future: session_id, capabilities, role
}
```

3 Command Definitions

3.1 Command Format

Every application-layer command is a variable-length byte sequence. The first byte is the command identifier; remaining bytes are command-specific parameters.

Field	Size	Description
Command ID	1 byte	Identifies the command type
Parameters	variable	Command-specific payload

3.2 CMD_UNLOCK (`0x01`)

Requests that the lock mechanism be actuated. This is the only command defined in the current revision.

Field	Size	Value
Command ID	1 byte	`0x01`
Parameters	0 bytes	No parameters in this revision (future: may carry unlock mode, duration, etc.)

3.2.1 Response

Field	Size	Value
Response Code	1 byte	`0x00` = success, `0x01` = denied, `0x02` = error
Details	0–N bytes	Human-readable or machine-parseable detail (optional; empty on success)

3.2.2 Processing

1. Extract `session_context.peer_identity_pk` and verify the peer has been authenticated by the session layer.
2. Apply authorization policy (Section 4).
3. If authorized, actuate the physical lock mechanism and return `0x00` (success).
4. If denied, return `0x01` without actuating.
5. If an internal error occurs (e.g. lock mechanism unresponsive), return `0x02`.

3.3 Reserved Command Space

The command ID namespace is allocated as follows:

Range	Purpose
`0x00`	Reserved (invalid command)
`0x01`	`CMD_UNLOCK`
`0x02--0x0F`	Reserved for future lock control commands
`0x10--0x1F`	Reserved for future OTA / firmware commands
`0x20--0x2F`	Reserved for future BLE / companion commands
`0x30--0x3F`	Reserved for future access control / provisioning commands
`0x40--0xEF`	Reserved
`0xF0--0xFF`	Reserved for protocol extension / vendor-specific

An unknown command ID MUST return response code `0x02` (error).

4 Authorization Policy

4.1 Current Model

In this revision, authentication implies authorization. Any peer that successfully completes the session layer's handshake and presents a provisioned Ed25519 public key is granted all commands including `CMD_UNLOCK`.

```
fn authorize(peer_identity_pk, command) -> bool {
    // Current revision: authenticated = authorized
    return is_provisioned(peer_identity_pk);  // always true for
       provisioned peers
}
```

4.2 Planned Extensions

See Section 6 for the authorization model roadmap.

5 System Architecture

5.1 Components

Component	Platform	Role
Mobile Application	Flutter (iOS/Android)	Initiates NFC session, sends encrypted unlock commands through the session layer
Lock Controller	ESP32 (ESP-IDF)	Verifies phone identity (via session layer), runs application-layer command dispatch, actuates lock mechanism

5.2 Cryptographic Libraries

Platform	Library	Responsibility
Flutter	`cryptography`	Ed25519 sign/verify, X25519 agreement, HKDF-SHA256, AES-256-GCM, secure RNG
Flutter	`flutter_secure_storage`	Storage of Ed25519 private identity key (Keychain / Keystore)
Flutter	`flutter_nfc_kit` / `nfc_manager`	NFC transport only – no cryptographic operations
ESP32	libsodium	Ed25519 sign/verify, X25519 agreement, secure RNG; key storage today, ATECC608A-backed in future revision
ESP32	mbedTLS (ESP-IDF)	HKDF-SHA256, AES-256-GCM

No custom cryptographic primitives are implemented on either platform.

6 Explicitly Deferred Items

The following items are acknowledged gaps, intentionally deferred rather than overlooked, and should be tracked for future revisions of this specification.

6.1 Authorization Model

Authentication currently implies authorization. Future work should introduce:

- An access control list (ACL) of authorized Phone public keys per Lock, or a signed capability/credential issued by a backend authority
- A revocation mechanism (e.g. denylist, short-lived signed credentials, or online revocation check) so a lost or decommissioned phone can be removed without re-provisioning the Lock's entire trust store

- Optional role/scope differentiation (e.g. owner vs. guest vs. time-limited access)

6.2 OTA / Firmware Update Command

A future `CMD_OTA_BEGIN` command would allow an authenticated and authorized peer to initiate a firmware update over the secure session. The application layer would validate the peer's authorization to perform updates (which may be more restrictive than unlock authorization), then coordinate with a platform-specific firmware update module. The session layer remains unchanged.

6.3 BLE Command Passthrough

When a BLE transport is added, the same application-layer command set and authorization policy apply. The application layer receives plaintext from the session layer regardless of whether it arrived over NFC or BLE. A future `CMD_BLE_*` range allows BLE-specific commands (e.g. status polling, configuration read-back) to be defined without modifying the core command dispatch.

6.4 Access Control and ACL Management

Commands for managing the access control list itself – provisioning a new phone identity, revoking an existing one, setting role attributes – are allocated the `0x30--0x3F` command ID range. These commands require stronger authorization than a standard unlock and will likely be restricted to an "owner" role.

7 Revision History

Version	Change	Date
0.1	**Rewritten as application-layer-only specification.** Extracted all cryptographic session content into *smartlock_session_layer.pdf*. Defines plaintext command byte format, `CMD_UNLOCK`, response codes, command ID namespace with reserved ranges for OTA/BLE/ACL, authorization policy, and the `on_command(plaintext, session_context)` abstract interface. No crypto awareness.	July 24, 2026